

Based on *Python Crash Course*

PYTHON MASTERY NOTES

PART 1 — CHAPTER 1 + CHAPTER 2

CHAPTER 1 — FIRST PYTHON THINKING

(Setup skipped as requested.)

This chapter is about understanding how Python works at the most basic level.

1. CONCEPT BREAKDOWN

What is Python?

Python is a programming language.

A programming language is a way to give instructions to a computer.

Example:

You tell a human:

"Bring me water."

You tell Python:

```
print("Hello")
```

Python reads your instruction and does the work.

What is a Program?

A program is a collection of instructions.

Example:

```
print("Hello")
print("How are you?")
print("Bye")
```

This is a program.

Python reads line by line.

What is Source Code?

The code you write is called source code.

Example:

```
name = "John"
print(name)
```

This code is source code.

What is Execution?

Execution means running the code.

When Python executes code:

- it reads line by line
- understands instructions
- performs actions

Example:

```
print("A")
print("B")
```

Output:

```
A
B
```

Python runs top to bottom.

What is Output?

Output means the result shown after code runs.

Example:

```
print("Hello")
```

Output:

```
Hello
```

What is print()?

`print()` is a built-in Python function.

Its job:

show output on screen.

Example:

```
print("Python")
```

Output:

```
Python
```

What is a Function?

A function is reusable code that performs a job.

Example:

`print()` is a function.

Its job:

display something.

Think:

Function = machine

Input goes in → work happens → output comes out

What is a String?

A string is text data.

Text must be inside quotes.

Example:

```
"Hello"  
"Python"  
"My name is Alex"
```

These are strings.

Why Quotes?

Because Python must know:

Is this text?

Or variable name?

Wrong:

```
print(Hello)
```

Python thinks `Hello` is a variable.

Correct:

```
print("Hello")
```

Quotes tell Python:

```
"This is text."
```

2. SYNTAX & MECHANICS

Look at this:

```
print("Hello World")
```

Breakdown:

print

Function name

Meaning:
call the print function

()

Parentheses

Used to pass information into a function

"Hello World"

String argument

Argument = data given to a function

Entire meaning

"Run print function and display Hello World"

Python Reads Top to Bottom

Example:

```
print("One")  
print("Two")  
print("Three")
```

Output:

```
One  
Two  
Three
```

Python follows order.

Case Sensitivity

Python cares about uppercase and lowercase.

Example:

```
print("hello")
```

works

But:

```
Print("hello")
```

Error.

Because:

```
print ≠ Print
```

Syntax Errors

Syntax = grammar rules of code.

Wrong grammar → error.

Example:

Wrong:

```
print("Hello)
```

Missing closing quote.

Python error.

Correct:

```
print("Hello")
```

3. PRACTICAL EXAMPLES

Example 1

```
print("I am learning Python")
```

Explanation:

- call print function
 - pass text
 - Python shows text
-

Example 2

```
print("My name is David")  
print("I love coding")
```

Explanation:

First line runs.

Then second line runs.

Example 3

```
print("Line 1")  
print("Line 2")  
print("Line 3")
```

Output:

Line 1
Line 2
Line 3

4. PRACTICE PROBLEMS

Do yourself first.

Problem 1

Print your name.

Problem 2

Print your age.

Problem 3

Print 3 lines:

- I am learning Python
 - I will become a programmer
 - I will practice daily
-

Problem 4

Create a syntax error intentionally.

Then fix it.

5. ANSWERS

Answer 1

```
print("Surjyo")
```

Answer 2

```
print("20")
```

Answer 3

```
print("I am learning Python")
print("I will become a programmer")
print("I will practice daily")
```

Answer 4

Wrong:

```
print("Hello)
```

Error because closing quote missing.

Correct:

```
print("Hello")
```

IMPORTANT TAKEAWAYS

Remember:

- Python executes line by line
 - `print()` shows output
 - strings need quotes
 - Python is case-sensitive
 - syntax must be exact
-
-

CHAPTER 2 — VARIABLES AND SIMPLE DATA TYPES

This chapter is VERY important.

If this chapter is weak, future chapters become hard.

Study carefully.

1. CONCEPT BREAKDOWN

What is a Variable?

A variable is a named container for storing data.

Think:

Bottle label → contains water

Variable name → contains data

Example:

```
name = "Alex"
```

Meaning:

Store "Alex" in variable called `name`

Variable Structure

```
name = "Alex"
```

Breakdown:

`name` → variable name

`=` → assignment operator

`"Alex"` → value

Meaning:

Put value into variable

Why Variables?

Without variables:

```
print("Alex")  
print("Alex")  
print("Alex")
```

With variable:

```
name = "Alex"  
print(name)  
print(name)  
print(name)
```

Better because reusable.

What is Assignment?

Assignment means storing value.

Example:

```
age = 21
```

Store 21 inside age.

Reassigning Variables

Variables can change.

Example:

```
age = 20  
age = 25
```

Final value = 25

Because latest assignment replaces old one.

Naming Rules

Good:

```
user_name  
student_age  
total_price
```

Bad:

```
2name  
user-name  
class
```

Rules:

- can contain letters
- numbers
- underscore

Cannot:

- start with number
 - use spaces
 - use reserved keywords
-

Data Types

Data type = kind of data.

Main beginner types:

- string
- integer
- float
- boolean

STRING

String = text

Examples:

```
"hello"  
"name"  
"python"
```

INTEGER

Whole number

Examples:

```
5  
100  
-20  
0
```

No decimal.

FLOAT

Decimal number

Examples:

```
3.14  
10.5  
99.99
```

BOOLEAN

Only two values:

```
True
```

False

Used for logic.

Example:

```
is_logged_in = True
```

2. STRING METHODS

Method = function attached to an object.

Example:

```
name.upper()
```

String has built-in methods.

upper()

Makes uppercase

```
name = "alex"
print(name.upper())
```

Output:

```
ALEX
```

lower()

Makes lowercase

```
name = "ALEX"
print(name.lower())
```

Output:

```
alex
```

title()

Capitalize words

```
name = "alex john"  
print(name.title())
```

Output:

```
Alex John
```

String Concatenation

Concatenation = joining strings

Example:

```
first = "John"  
last = "Smith"  
  
full = first + last
```

Problem:

Output:

```
JohnSmith
```

Need space:

```
full = first + " " + last
```

Now:

```
John Smith
```

Whitespace

Whitespace means invisible spacing.

Includes:

- spaces

- tabs
- new lines

Newline

`\n`

Moves next line.

Example:

```
print("Hello\nWorld")
```

Output:

```
Hello
World
```

Tab

`\t`

Adds tab space.

Example:

```
print("A\tB")
```

Numbers

Python can calculate.

Example:

```
2 + 3
5 - 1
4 * 2
10 / 2
```

Operators:

Addition:

+

Subtraction:

-

Multiplication:

*

Division:

/

Exponent:

**

Example:

`2 ** 3`

Output:

8

Type Conversion

Sometimes Python needs conversion.

Example:

Wrong:

```
age = 20
message = "Age is " + age
```

Error.

Why?

String + integer cannot combine directly.

Fix:

```
message = "Age is " + str(age)
```

`str()` converts to string.

Comments

Comments are notes for humans.

Python ignores comments.

Example:

```
# store user age  
age = 20
```

2. SYNTAX & MECHANICS

Variable:

```
name = "Alex"
```

Mechanics:

Left side = variable name

Right side = value

= means assign

Method call:

```
name.upper()
```

Mechanics:

```
variable.method()
```

Conversion:

```
str(age)
```

Mechanics:

```
function(value)
```

3. PRACTICAL EXAMPLES

Example 1

```
name = "surjyo"  
print(name)
```

Explanation:

Create variable

Store text

Print variable

Example 2

```
name = "surjyo"  
print(name.upper())
```

Output:

SURJYO

Example 3

```
age = 20  
print(age + 5)
```

Output:

25

Example 4

```
first = "John"
last = "Doe"

full = first + " " + last

print(full)
```

Output:

John Doe

Example 5

```
price = 99.99
tax = 10

total = price + tax

print(total)
```

4. PRACTICE PROBLEMS

Do first.

Problem 1

Create variable `name`.

Store your name.

Print it.

Problem 2

Create variable `age`.

Print:

Age is 20

(using variable)

Problem 3

Convert name to uppercase.

Problem 4

Create two variables:

first_name

last_name

Join them.

Problem 5

Calculate:

15 + 10

50 - 20

6 * 7

100 / 4

Problem 6

Use newline to print:

Python

Java

C++

5. ANSWERS

Practice first before reading.

Answer 1

```
name = "Surjyo"  
print(name)
```

Answer 2

```
age = 20  
print("Age is " + str(age))
```

Why str?

Because age is integer.

Text + integer causes error.

Answer 3

```
name = "surjyo"  
print(name.upper())
```

Answer 4

```
first_name = "John"  
last_name = "Smith"  
  
full_name = first_name + " " + last_name  
  
print(full_name)
```

Answer 5

```
print(15 + 10)  
print(50 - 20)  
print(6 * 7)  
print(100 / 4)
```

Answer 6

```
print("Python\nJava\nC++")
```

COMMON BEGINNER MISTAKES

Wrong:

```
name = Surjyo
```

Problem:

No quotes.

Python thinks variable.

Correct:

```
name = "Surjyo"
```

Wrong:

```
2name = "Alex"
```

Cannot start with number.

Wrong:

```
"Age: " + 20
```

Type mismatch.

Fix:

```
"Age: " + str(20)
```

PRO TIPS

Learn these deeply:

- variables
- strings
- numbers
- methods
- type conversion

These are used EVERYWHERE in Python.

PYTHON MASTERY NOTES

PART 2 — CHAPTER 3 + CHAPTER 4

Based on *Python Crash Course*

This section is EXTREMELY important.

If variables are the foundation, then **lists + loops are the muscles of Python**.

Most real programs use them constantly.

Read carefully.

Type every code yourself.

CHAPTER 3 — LISTS

1. CONCEPT BREAKDOWN

What is a List?

A list is a container that stores multiple values in one variable.

Example:

Instead of:


```
student1 = "Alex"  
student2 = "John"  
student3 = "Emma"
```

Better:

```
students = ["Alex", "John", "Emma"]
```

One variable.

Multiple values.

Think of a list like a backpack.

Inside one bag:

- notebook
- pen
- charger
- phone

Same idea.

Python list:

- one container
 - many items
-

Syntax of a List

```
names = ["Alex", "John", "Emma"]
```

Breakdown:

names → variable name

= → assignment

[] → list container

Items inside:

```
"Alex"  
"John"  
"Emma"
```

Comma separates items.

Why Use Lists?

Because they make data management easier.

Without list:

messy

With list:

organized

Reusable

Loop-friendly

Easy to modify

Accessing List Items

Every item has a position.

Python starts counting from ZERO.

VERY IMPORTANT.

Example:

```
names = ["Alex", "John", "Emma"]
```

Positions:

```
Alex → 0  
John → 1  
Emma → 2
```

Access:

```
print(names[0])
```

Output:

Alex

Why Start From 0?

Because Python indexing starts at zero.

Do not fight this.

Accept it early.

It becomes natural.

Negative Indexing

Python can count backward.

Example:

```
names = ["Alex", "John", "Emma"]
```

Positions:

Emma → -1

John → -2

Alex → -3

Example:

```
print(names[-1])
```

Output:

Emma

Useful when you want last item.

Modifying List Items

Lists are mutable.

Mutable means changeable.

Example:

```
names = ["Alex", "John", "Emma"]  
names[1] = "David"
```

Now:

```
["Alex", "David", "Emma"]
```

Adding Items

append()

Adds item at END.

Example:

```
names = ["Alex", "John"]  
names.append("Emma")
```

Result:

```
["Alex", "John", "Emma"]
```

insert()

Add item at specific position.

Example:

```
names.insert(1, "David")
```

Result:

```
["Alex", "David", "John"]
```

Removing Items

del

Deletes by position.

Example:

```
del names[1]
```

Removes item permanently.

pop()

Removes item AND returns it.

Example:

```
last = names.pop()
```

Removes last item.

Stores removed item.

Why useful?

Example:

Shopping cart:

Remove item but remember what user removed.

remove()

Remove by VALUE.

Example:

```
names.remove("John")
```

Removes John.

Not by index.

By actual value.

Sorting Lists

sort()

Permanent sorting.

Example:

```
numbers = [5, 2, 9, 1]  
numbers.sort()
```

Result:

```
[1, 2, 5, 9]
```

Original changed forever.

Descending:

```
numbers.sort(reverse=True)
```

sorted()

Temporary sorting.

Original unchanged.

Example:

```
numbers = [5, 2, 9]
```

```
print(sorted(numbers))  
print(numbers)
```

Output:

```
[2, 5, 9]  
[5, 2, 9]
```

Reverse

```
numbers.reverse()
```

Reverses current order.

Length

```
len(names)
```

Returns number of items.

Example:

```
names = ["A", "B", "C"]  
  
print(len(names))
```

Output:

```
3
```

2. SYNTAX & MECHANICS

List creation:

```
items = [1, 2, 3]
```

Access:

```
items[0]
```

Mechanics:

```
variable[index]
```

Change:

```
items[0] = 100
```

Mechanics:

```
list[index] = new_value
```

Append:

```
items.append(50)
```

Mechanics:

```
list.method(value)
```

Insert:

```
items.insert(position, value)
```

Pop:

```
items.pop()
```

Remove:

```
items.remove(value)
```

3. PRACTICAL EXAMPLES

Example 1

```
fruits = ["apple", "banana", "mango"]
```



```
print(fruits[0])
```

Output:

```
apple
```

Example 2

```
fruits = ["apple", "banana"]
```

```
fruits.append("orange")
```

```
print(fruits)
```

Output:

```
['apple', 'banana', 'orange']
```

Example 3

```
fruits = ["apple", "banana", "mango"]
```

```
fruits.remove("banana")
```

```
print(fruits)
```

Output:

```
['apple', 'mango']
```

Example 4

```
numbers = [10, 3, 7, 1]
```

```
numbers.sort()
```

```
print(numbers)
```

PRACTICE PROBLEMS

1. Create a list of 5 countries.
2. Print first country.

3. Print last country.
 4. Replace second country.
 5. Add new country.
 6. Remove one country.
 7. Sort alphabetically.
-

ANSWERS

```
countries = ["USA", "Japan", "India", "Canada", "Germany"]  
  
print(countries[0])  
  
print(countries[-1])  
  
countries[1] = "Brazil"  
  
countries.append("France")  
  
countries.remove("India")  
  
countries.sort()  
  
print(countries)
```

COMMON MISTAKES

Wrong:

```
names[10]
```

Error if list has only 3 items.

Wrong:

```
names.remove("xyz")
```

Error if item doesn't exist.

CHAPTER 4 — WORKING WITH LISTS

Now we move to POWER tools.

This chapter teaches loops, range, slicing, comprehensions, tuples.

These are used constantly.

What is a Loop?

A loop repeats code.

Without loop:

```
print("Hello Alex")
print("Hello John")
print("Hello Emma")
```

Bad.

With loop:

```
for name in names:
    print(name)
```

Better.

Think:

Robot repeating instructions.

for Loop

Example:

```
names = ["Alex", "John", "Emma"]
```

```
for name in names:  
    print(name)
```

Meaning:

Take each item from list.

Store temporarily in variable `name`.

Run code.

Repeat.

Step by step:

First:

```
name = "Alex"
```

Print.

Second:

```
name = "John"
```

Print.

Third:

```
name = "Emma"
```

Print.

Done.

Indentation

VERY IMPORTANT.

Python uses indentation to define code blocks.

Correct:

```
for name in names:  
    print(name)
```

Wrong:

```
for name in names:  
print(name)
```

Error.

range()

Generates sequence of numbers.

Example:

```
range(5)
```

Gives:

```
0, 1, 2, 3, 4
```

NOT 5.

Example:

```
for i in range(5):  
    print(i)
```

Output:

```
0  
1  
2  
3  
4
```

range(start, stop)

Example:

```
range(2, 6)
```

Gives:

```
2, 3, 4, 5
```

range(start, stop, step)

Example:

```
range(0, 10, 2)
```

Gives:

```
0, 2, 4, 6, 8
```

list()

Convert iterable into list.

Example:

```
numbers = list(range(5))
```

Result:

```
[0, 1, 2, 3, 4]
```

Simple Statistics

Functions:

```
min()  
max()  
sum()
```

Example:

```
numbers = [10, 20, 30]
```

```
print(min(numbers))  
print(max(numbers))  
print(sum(numbers))
```

Output:

```
10
30
60
```

List Comprehension

Advanced but important.

Compact list creation.

Normal:

```
squares = []

for x in range(5):
    squares.append(x * x)
```

Short version:

```
squares = [x * x for x in range(5)]
```

Same result.

Breakdown:

`x * x` → what to create

`for x in range(5)` → loop source

Slicing

Get part of list.

Example:

```
names = ["A", "B", "C", "D", "E"]

print(names[1:4])
```

Output:

```
['B', 'C', 'D']
```

Rule:

Start included.

End excluded.

Examples:

First 3:

```
names[:3]
```

From index 2 onward:

```
names[2:]
```

Copy full list:

```
names[:]
```

Why Copy Matters

Bad:

```
list2 = list1
```

Both point same list.

Changing one changes both.

Safer:

```
list2 = list1[:]
```

True copy.

Tuples

Tuple = immutable list.

Immutable = cannot change.

Example:

```
dimensions = (200, 50)
```

Uses:

- fixed values
 - coordinates
 - settings
-

Access:

```
dimensions[0]
```

Works.

Change:

```
dimensions[0] = 300
```

Error.

List vs Tuple

List:

changeable

Tuple:

not changeable

2. SYNTAX & MECHANICS

Loop:

```
for item in items:  
    print(item)
```

Mechanics:

for

loop keyword

item

temporary variable

in

take from collection

items

source

:

start block

indented code = loop body

Slicing:

```
list[start:end]
```

Tuple:

```
(1, 2, 3)
```

PRACTICAL EXAMPLES

Example 1

```
numbers = [1, 2, 3]

for num in numbers:
    print(num)
```

Example 2

```
for i in range(1, 6):
    print(i)
```

Example 3

```
nums = [10, 20, 30, 40]

print(nums[1:3])
```

Output:

```
[20, 30]
```

Example 4

```
squares = [x*x for x in range(10)]

print(squares)
```

HIGH-YIELD PRACTICE

1. Print numbers 1 to 20.
 2. Print even numbers 2 to 20.
 3. Create squares list.
 4. Find sum of numbers.
 5. Slice first 5 items.
 6. Copy list and modify copy.
 7. Create tuple of days.
-

ANSWERS

1

```
for i in range(1, 21):  
    print(i)
```

2

```
for i in range(2, 21, 2):  
    print(i)
```

3

```
squares = [x*x for x in range(1, 11)]
```

4

```
nums = [1, 2, 3, 4, 5]  
print(sum(nums))
```

5

```
items[:5]
```

6

```
copy_items = items[:]
```

7

```
days = ("Mon", "Tue", "Wed")
```

PRO TIPS

Master these fully:

- indexing
- append
- remove
- pop
- sort
- loops
- range
- slicing
- list comprehension
- tuples

These are everywhere in Python.

PYTHON MASTERY NOTES

PART 3 — CHAPTER 5 + CHAPTER 6

Based on *Python Crash Course*

This section changes how you think.

Until now, your code only followed fixed instructions.

Now your code starts **making decisions**.

That is real programming.

CHAPTER 5 — IF STATEMENTS (DECISION MAKING)

1. CONCEPT BREAKDOWN

What is an if Statement?

An `if` statement lets your program make decisions.

Think:

Real life:

"If it is raining, take umbrella."

Programming:

"If condition is true, run code."

Example:

```
age = 20

if age >= 18:
    print("Adult")
```

Meaning:

If age is greater than or equal to 18, print Adult.

What is a Condition?

A condition is a test.

It asks:

True or False?

Example:

```
age >= 18
```

Question:

Is age greater than or equal to 18?

Answer:

- True
- or
- False

Boolean Values

Python decision making uses booleans.

Only two:

```
True
False
```

Example:

```
10 > 5
```

Result:

```
True
```

Example:

```
2 > 9
```

Result:

```
False
```

Comparison Operators

These compare values.

Equal:

```
==
```

Not equal:

```
!=
```

Greater than:

>

Less than:

<

Greater or equal:

>=

Less or equal:

<=

Examples:

5 == 5

True

10 != 3

True

4 < 2

False

IMPORTANT: = vs ==

Very common beginner mistake.

Single equals:

=

Means assignment.

Example:

age = 20

Store value.

Double equals:

==

Means comparison.

Example:

```
age == 20
```

Check equality.

DO NOT confuse these.

Simple if

Example:

```
age = 20

if age >= 18:
    print("Adult")
```

Meaning:

If condition true → run code.

If false → skip.

if-else

What if condition is false?

Use else.

Example:

```
age = 15

if age >= 18:
    print("Adult")
else:
```

```
print("Minor")
```

Meaning:

If true → Adult

Otherwise → Minor

if-elif-else

Multiple conditions.

Example:

```
score = 85

if score >= 90:
    print("A")
elif score >= 80:
    print("B")
else:
    print("Below B")
```

Meaning:

Check top condition first.

If false → check next.

Continue until match.

Think like security gate.

Check first.

If fail → next.

Logical Operators

Used for combining conditions.

AND:

and

Both must be true.

Example:

```
age > 18 and country == "USA"
```

OR:

or

At least one true.

Example:

```
age > 18 or has_permission == True
```

NOT:

not

Reverses result.

Example:

```
not True
```

Result:

```
False
```

Membership Testing

Check if item exists.

in

Example:

```
fruits = ["apple", "banana"]
```

```
"apple" in fruits
```

True

Not in:

```
not in
```

Example:

```
"mango" not in fruits
```

True

Empty Lists

Empty list:

```
[]
```

Python treats empty collections as False.

Example:

```
users = []

if users:
    print("Users exist")
else:
    print("No users")
```

2. SYNTAX & MECHANICS

Simple if:

```
if condition:
    code
```

Mechanics:

```
if
```

decision keyword

condition

boolean expression

:

start block

indentation

code inside block

if else:

```
if condition:
    code1
else:
    code2
```

if elif else:

```
if condition1:
    code
elif condition2:
    code
else:
    code
```

AND:

a and b

Both true.

OR:

a or b

At least one true.

3. PRACTICAL EXAMPLES

Example 1

```
age = 20

if age >= 18:
    print("You can vote")
```

Example 2

```
age = 14

if age >= 18:
    print("Adult")
else:
    print("Minor")
```

Example 3

```
marks = 72

if marks >= 90:
    print("A")
elif marks >= 75:
    print("B")
elif marks >= 60:
    print("C")
else:
    print("Fail")
```

Example 4

```
username = "admin"
password = "1234"

if username == "admin" and password == "1234":
    print("Login success")
```

Example 5

```
fruits = ["apple", "banana"]

if "apple" in fruits:
```

```
print("Available")
```

HIGH-YIELD PRACTICE

1. Check if age is adult.
 2. Compare two numbers.
 3. Build grade checker.
 4. Login checker.
 5. Check if username exists in list.
 6. Check empty cart.
 7. ATM withdrawal permission checker.
-

ANSWERS

1

```
age = 22

if age >= 18:
    print("Adult")
else:
    print("Minor")
```

2

```
a = 10
b = 20

if a > b:
    print("A bigger")
else:
    print("B bigger")
```

3

```
marks = 88

if marks >= 90:
    print("A")
elif marks >= 80:
    print("B")
elif marks >= 70:
```

```
    print("C")
else:
    print("Fail")
```

COMMON MISTAKES

WRONG:

```
if age = 18:
```

Error.

Because assignment used.

Correct:

```
if age == 18:
```

WRONG indentation:

```
if age > 18:
print("Adult")
```

Correct:

```
if age > 18:
    print("Adult")
```

CHAPTER 6 — DICTIONARIES

This is a MASSIVE topic.

Lists store values.

Dictionaries store key-value pairs.

This is how real apps store structured data.

1. CONCEPT BREAKDOWN

What is a Dictionary?

A dictionary stores data in pairs:

key → value

Example:

Real dictionary:

word → meaning

Python dictionary:

name → Alex

age → 20

country → Japan

Example:

```
user = {  
    "name": "Alex",  
    "age": 20,  
    "country": "Japan"  
}
```

Think:

Label → value

"name" is label.

"Alex" is data.

Why Dictionaries?

Lists are position-based:

```
student[0]
```

Hard to remember.

Dictionary:

```
student["name"]
```

Much clearer.

Dictionary Structure

```
{  
    key: value  
}
```

Example:

```
{  
    "name": "Alex"  
}
```

Accessing Values

Example:

```
user = {  
    "name": "Alex",  
    "age": 20  
}
```

```
print(user["name"])
```

Output:

```
Alex
```

Adding New Data

Example:

```
user["email"] = "alex@gmail.com"
```

Now dictionary grows.

Modifying Data

Example:

```
user["age"] = 25
```

Old value replaced.

Removing Data

Using del:

```
del user["age"]
```

Deletes key and value.

Empty Dictionary

Example:

```
user = {}
```

Useful when building data step by step.

Looping Through Dictionary

items()

Get key + value.

Example:

```
for key, value in user.items():  
    print(key, value)
```

keys()

Only keys.

Example:

```
for key in user.keys():  
    print(key)
```

values()

Only values.

Example:

```
for value in user.values():  
    print(value)
```

Nesting

Advanced but important.

Means storing structures inside structures.

Dictionary inside list:

```
users = [  
    {"name": "Alex"},  
    {"name": "John"}  
]
```

List inside dictionary:

```
student = {  
    "name": "Alex",  
    "skills": ["Python", "SQL"]  
}
```

Dictionary inside dictionary:

```
users = {  
    "user1": {"name": "Alex"},  
    "user2": {"name": "John"}  
}
```

get()

Safe access.

Problem:

```
print(user["salary"])
```

Error if key missing.

Safer:

```
print(user.get("salary"))
```

Returns None instead of crashing.

With default:

```
print(user.get("salary", "Not found"))
```

2. SYNTAX & MECHANICS

Create:

```
data = {  
    "key": "value"  
}
```

Access:

```
data["key"]
```

Add:

```
data["new_key"] = value
```

Update:

```
data["key"] = new_value
```

Delete:

```
del data["key"]
```

Loop:

```
for k, v in data.items():
```

3. PRACTICAL EXAMPLES

Example 1

```
student = {  
    "name": "Surjyo",  
    "age": 20  
}  
  
print(student["name"])
```

Example 2

```
student["country"] = "Bangladesh"
```

Example 3

```
student["age"] = 21
```

Example 4

```
for key, value in student.items():  
    print(key, value)
```

Example 5

```
product = {  
    "name": "Laptop",  
    "price": 500  
}  
  
print(product.get("stock", "Unknown"))
```

HIGH-YIELD PRACTICE

1. Create student dictionary.
 2. Add email.
 3. Update age.
 4. Delete one field.
 5. Loop all keys.
 6. Loop all values.
 7. Build mini contact book.
 8. Build product inventory dictionary.
-

ANSWERS

Example solution

```
student = {  
    "name": "Surjyo",  
    "age": 20  
}  
  
student["email"] = "test@gmail.com"  
  
student["age"] = 21  
  
del student["email"]  
  
for key in student.keys():  
    print(key)  
  
for value in student.values():  
    print(value)
```

COMMON MISTAKES

WRONG:

```
user[name]
```

Python thinks variable.

Correct:

```
user["name"]
```

WRONG:

```
user["salary"]
```

Error if missing.

Better:

```
user.get("salary")
```

PRO TIPS

Master deeply:

- if
- elif
- else
- comparisons
- boolean logic
- in / not in
- dictionaries
- get()
- items()
- nesting

These are used in interviews and real projects constantly.

PYTHON MASTERY NOTES

PART 4 — CHAPTER 7 + CHAPTER 8

Based on *Python Crash Course*

This is where you stop writing static programs.

Now your programs become interactive and reusable.

This part is VERY important.

If you master functions + loops, your coding level jumps a lot.

CHAPTER 7 — USER INPUT + WHILE LOOPS

1. CONCEPT BREAKDOWN

What is User Input?

User input means your program asks the user for information.

Example:

A program asks:

“What is your name?”

User types:

Alex

Program uses that data.

Without input:

program is fixed

With input:

program becomes interactive

input()

Python uses `input()` to get data from users.

Example:

```
name = input("Enter your name: ")
print(name)
```

Meaning:

- show message
 - wait for user typing
 - store typed value
 - continue program
-

Important Truth About input()

`input()` returns TEXT (string).

Even if user types a number.

Example:

User types:

25

Python stores:

"25"

NOT integer.

VERY important.

Why This Matters

Wrong:

```
age = input("Enter age: ")  
print(age + 5)
```

Error.

Because:

string + integer

Not allowed.

Fix:

```
age = int(input("Enter age: "))  
print(age + 5)
```

Now input becomes integer.

Type Conversion Review

Convert to integer:

```
int()
```

Convert to string:

```
str()
```

Convert to decimal:

```
float()
```

What is a while Loop?

A `while` loop repeats code while condition remains true.

Example:

```
count = 1

while count <= 5:
    print(count)
    count += 1
```

Output:

```
1
2
3
4
5
```

Meaning:

Check condition.

If true → run code.

Check again.

Repeat.

Stop when false.

Difference: for vs while

FOR:

use when you know how many times.

Example:

Print 1 to 10.

WHILE:

use when repetition depends on condition.

Example:

Keep asking password until correct.

Loop Variable

Example:

```
count = 1
```

This controls loop.

Without changing it:

loop never ends.

Updating Variables

Example:

```
count += 1
```

Means:

```
count = count + 1
```

Very common.

Infinite Loop

Dangerous mistake.

Example:

```
count = 1
```

```
while count <= 5:  
    print(count)
```

Problem:

count never changes.

Condition always true.

Loop runs forever.

break

Immediately exits loop.

Example:

```
while True:
    text = input("Type quit to stop: ")

    if text == "quit":
        break
```

Meaning:

Leave loop immediately.

continue

Skip current iteration.

Example:

```
for i in range(5):
    if i == 2:
        continue

    print(i)
```

Output:

```
0
1
3
4
```

2 skipped.

Flags

A flag is a control variable.

Usually boolean.

Example:

```
active = True
```

Loop:

```
while active:
```

Change later:

```
active = False
```

Loop stops.

Cleaner design.

Modulo Operator %

Returns remainder.

Example:

```
10 % 3
```

Result:

```
1
```

Because:

$10 \div 3 = 3$ remainder 1

Useful for even/odd:

```
number % 2 == 0
```

Even if remainder zero.

2. SYNTAX & MECHANICS

Input:

```
input("message")
```

Integer input:

```
int(input("message"))
```

While loop:

```
while condition:
    code
```

Break:

```
break
```

Continue:

```
continue
```

Increment:

```
x += 1
```

3. PRACTICAL EXAMPLES

Example 1 — Name Input

```
name = input("Enter name: ")
print("Hello " + name)
```

Example 2 — Age Calculator

```
age = int(input("Enter age: "))
print(age + 10)
```

Example 3 — Counting Loop

```
count = 1

while count <= 5:
    print(count)
    count += 1
```

Example 4 — Password Checker

```
password = ""

while password != "python123":
    password = input("Enter password: ")

print("Access granted")
```

Step:

Start with empty password.

Condition true.

Ask user.

Repeat until correct.

Example 5 — Even Number Checker

```
num = int(input("Enter number: "))

if num % 2 == 0:
    print("Even")
else:
    print("Odd")
```

HIGH-YIELD PRACTICE

1. Ask user name and greet.
 2. Ask age and calculate future age.
 3. Print numbers 1–20 using while.
 4. Password checker.
 5. ATM PIN checker.
 6. Number guessing loop.
 7. Even/odd checker.
 8. Shopping cart input system.
-

ANSWERS

Example

```
count = 1

while count <= 10:
    print(count)
    count += 1
```

Password:

```
while True:
    pwd = input("Enter password: ")

    if pwd == "admin":
        break
```

COMMON MISTAKES

WRONG:

```
age = input()
print(age + 5)
```

String problem.

Fix:

```
age = int(input())
```

WRONG:

```
while x < 10:  
    print(x)
```

If x never changes → infinite loop.

CHAPTER 8 — FUNCTIONS

This chapter is HUGE.

Functions are one of the most important concepts in programming.

Master this deeply.

1. CONCEPT BREAKDOWN

What is a Function?

A function is reusable code that performs a task.

Without function:

```
print("Hello Alex")  
print("Hello John")  
print("Hello Emma")
```

Messy.

With function:

```
def greet(name):  
    print("Hello " + name)
```

Reusable.

Think:

Function = machine

Input goes in

Work happens

Output may come out

Why Functions?

Because they:

- reduce repetition
- organize code
- improve readability
- make debugging easier
- make reuse easy

Real software depends heavily on functions.

Defining a Function

Example:

```
def greet():  
    print("Hello")
```

Meaning:

Create function named greet.

Does not run yet.

Calling a Function

Run it:

```
greet()
```

Output:

```
Hello
```

Parameters

Parameters are placeholders inside function definition.

Example:

```
def greet(name):
```

name is parameter.

It waits for value.

Arguments

Actual values passed in.

Example:

```
greet("Alex")
```

"Alex" is argument.

Difference:

Parameter = placeholder

Argument = actual value

Very important.

Function with Parameters

Example:

```
def greet(name):  
    print("Hello " + name)
```

Call:

```
greet("Alex")
```

Output:

```
Hello Alex
```

Multiple Parameters

Example:

```
def add(a, b):  
    print(a + b)
```

Call:

```
add(5, 3)
```

Output:

```
8
```

Return Values

Some functions send data back.

Example:

```
def add(a, b):  
    return a + b
```

Meaning:

Give result back.

Use:

```
result = add(5, 3)
```

```
print(result)
```

Difference:

`print()`:

shows result

`return`:

sends result back

VERY important.

Example Difference

Bad design:

```
def add(a, b):  
    print(a + b)
```

Reusable? Less flexible.

Better:

```
def add(a, b):  
    return a + b
```

Now:

store result

reuse result

modify result

better design

Default Parameters

Example:

```
def greet(name="Guest"):  
    print("Hello " + name)
```

Call:

```
greet()
```

Output:

```
Hello Guest
```

Or:

```
greet("Alex")
```

Output:

```
Hello Alex
```

Positional Arguments

Order matters.

Example:

```
def info(name, age):
```

Call:

```
info("Alex", 20)
```

Meaning:

```
name = Alex
```

```
age = 20
```

Wrong order:

```
info(20, "Alex")
```

Bad logic.

Keyword Arguments

Explicit naming.

Example:

```
info(age=20, name="Alex")
```

Order no longer matters.

Arbitrary Arguments (*args)

Unknown number of values.

Example:

```
def total(*numbers):  
    print(numbers)
```

Call:

```
total(1, 2, 3, 4)
```

Python collects into tuple.

Arbitrary Keyword Arguments (**kwargs)

Unknown named values.

Example:

```
def profile(**data):  
    print(data)
```

Call:

```
profile(name="Alex", age=20)
```

Python creates dictionary.

Scope

Variable visibility.

Inside function:

local variable

Example:

```
def test():  
    x = 10
```

x exists only inside.

Outside:

error.

Global Variables

Created outside function.

Example:

```
name = "Alex"
```

Accessible globally (with care).

Modules

A module is a Python file containing reusable code.

Example:

math_tools.py

Contains:

```
def add(a, b):  
    return a + b
```

Use:

```
import math_tools
```

Call:

```
math_tools.add(2, 3)
```

Import Specific Function

```
from math_tools import add
```

Then:

```
add(2, 3)
```

Alias

Rename import.

Example:

```
import math_tools as mt
```

Use:

```
mt.add()
```

2. SYNTAX & MECHANICS

Function definition:

```
def function_name():  
    code
```

Parameter:

```
def greet(name):
```

Call:

```
greet("Alex")
```

Return:

```
return value
```

Default:

```
def test(x=10):
```

Arbitrary:

```
*args
```

Keyword arbitrary:

```
**kwargs
```

3. PRACTICAL EXAMPLES

Example 1

```
def greet():  
    print("Hello")
```

Call:

```
greet()
```

Example 2

```
def greet(name):  
    print("Hello " + name)
```

Example 3

```
def square(num):  
    return num * num
```

Use:

```
result = square(5)  
print(result)
```

Example 4

```
def is_even(num):  
    return num % 2 == 0
```

Example 5

```
def total(*numbers):  
    return sum(numbers)
```

HIGH-YIELD PRACTICE

1. Create greet function.
 2. Create add function.
 3. Create square function.
 4. Create login checker.
 5. Create even checker.
 6. Create calculator functions.
 7. Create profile function using kwargs.
 8. Create shopping cart total function.
-

ANSWERS

Add

```
def add(a, b):  
    return a + b
```

Square:

```
def square(x):  
    return x * x
```

Even:

```
def is_even(x):  
    return x % 2 == 0
```

Profile:

```
def profile(**data):  
    print(data)
```

COMMON MISTAKES

WRONG:

```
def greet
```

Missing:

```
()  
:
```

WRONG:

```
return
```

with no logic when expected.

Confusing print vs return.

This is HUGE beginner problem.

PRO TIPS

Master deeply:

- input
- int conversion
- while
- break
- continue
- modulo
- functions
- parameters
- arguments
- return
- scope
- modules

These are professional fundamentals.

PYTHON MASTERY NOTES

PART 5 — CHAPTER 9 + CHAPTER 10

Based on *Python Crash Course*

This is where beginners usually get scared.

Do NOT panic.

I will make this easy.

This part is where you start thinking like a real software developer.

Because now we learn:

- OOP (Object-Oriented Programming)
- Classes
- Objects
- Inheritance
- Files
- Error handling
- Exceptions

These are used in real applications everywhere.

CHAPTER 9 — CLASSES AND OBJECT-ORIENTED PROGRAMMING (OOP)

1. CONCEPT BREAKDOWN

What is OOP?

OOP means:

Object-Oriented Programming.

This is a way of organizing code.

Instead of writing random functions everywhere, we model real things.

Example:

Real world:

- Car
- Student
- Bank Account
- User
- Product

In Python:

We create code versions of these.

Think:

Blueprint → house

A blueprint describes the house.

Then many houses can be built from it.

Same in Python:

Class = blueprint

Object = actual created thing

What is a Class?

A class is a blueprint/template.

Example:

Car blueprint:

A car has:

- color
- brand
- speed

A car can:

- start
- stop
- accelerate

Python version:

```
class Car:  
    pass
```

This creates blueprint.

No actual car yet.

What is an Object?

Object = actual instance created from class.

Example:

```
my_car = Car()
```

Now object exists.

Think:

Class = cookie mold

Object = actual cookie

What is an Instance?

Object and instance are basically the same idea here.

Example:

```
user1 = User()
```

user1 is an instance.

What is a Method?

A method is a function inside a class.

Example:

```
class Dog:
    def bark(self):
        print("Woof")
```

bark() is a method.

Difference:

Normal function:

outside class

Method:

inside class

What is self?

This confuses beginners.

Simple explanation:

`self` means:

THIS object.

Example:

If object is:

`dog1`

then inside method:

`self` means `dog1`.

Think:

`self = me`

Constructor (init)

Special method.

Runs automatically when object is created.

Example:

```
class User:
    def __init__(self, name):
        self.name = name
```

When:

```
u = User("Alex")
```

Python automatically runs:

```
__init__()
```

Why use constructor?

To initialize object data.

Attributes

Variables inside objects.

Example:

```
self.name = name
```

Attribute:

```
name
```

Example:

A student object may have:

- name
- age
- roll

These are attributes.

Methods vs Attributes

Attribute:

stored data

Example:

```
self.name
```

Method:

behavior

Example:

```
def speak(self):
```

First Full Example

```
class Dog:
    def __init__(self, name):
        self.name = name

    def bark(self):
        print(self.name + " says woof")
```

Create object:

```
dog1 = Dog("Rocky")
```

Use:

```
dog1.bark()
```

Output:

```
Rocky says woof
```

Step by step:

Create class

Create constructor

Store name

Create bark method

Create object

Call method

Modifying Attributes

Example:

```
user.age = 25
```

Change stored value.

Default Attributes

Example:

```
self.login_attempts = 0
```

Starts with default value.

Inheritance

Huge concept.

Inheritance means:

child class gets parent features.

Example:

Parent:

Animal

Child:

Dog

Dog automatically gets animal abilities.

Example:

```
class Animal:
    def eat(self):
        print("Eating")
```

Child:

```
class Dog(Animal):
    pass
```

Now:

```
dog = Dog()
dog.eat()
```

Works.

Why inheritance?

Reuse code.

Cleaner design.

Less duplication.

Overriding Methods

Child can replace parent behavior.

Example:

Parent:

```
def speak():
    print("Sound")
```

Child:

```
def speak():
    print("Bark")
```

Child version wins.

Composition

Object inside object.

Example:

Car has Engine.

Instead of inheritance.

Example:

```
self.engine = Engine()
```

Real-world modeling.

2. SYNTAX & MECHANICS

Class:

```
class ClassName:
```

Constructor:

```
def __init__(self):
```

Attribute:

```
self.name = name
```

Method:

```
def method_name(self):
```

Create object:


```
obj = ClassName()
```

Call method:

```
obj.method()
```

Inheritance:

```
class Child(Parent):
```

3. PRACTICAL EXAMPLES

Example 1 — Student Class

```
class Student:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def introduce(self):
        print("My name is " + self.name)
```

Use:

```
s1 = Student("Surjyo", 20)
s1.introduce()
```

Example 2 — Bank Account

```
class BankAccount:
    def __init__(self, balance):
        self.balance = balance

    def deposit(self, amount):
        self.balance += amount

    def show_balance(self):
        print(self.balance)
```

Example 3 — Inheritance

```
class Animal:
    def speak(self):
```

```
        print("Animal sound")
class Dog(Animal):
    def speak(self):
        print("Woof")
```

HIGH-YIELD PRACTICE

1. Create Car class.
 2. Create User class.
 3. Create Product class.
 4. Build BankAccount class.
 5. Build ATM simulation.
 6. Create inheritance with Animal → Cat.
 7. Build Student management system.
-

COMMON BEGINNER MISTAKES

WRONG:

```
def bark():
```

Inside class must usually have:

```
self
```

WRONG:

```
dog = Dog
```

This stores class reference.

Need object:

```
dog = Dog()
```

WRONG:

forgetting `self`.

```
name = name
```

Correct:

```
self.name = name
```

CHAPTER 10 — FILES + EXCEPTIONS

This is extremely practical.

Real apps read files.

Real apps face errors.

Professionals handle both.

What is a File?

A file stores data permanently.

Examples:

- txt
- csv
- json
- log files

Without files:

data disappears after program ends.

Opening Files

Example:

```
with open("notes.txt") as file:  
    content = file.read()
```

Meaning:

Open file

Read content

Store data

Why with?

Because Python automatically closes file.

Safer.

Cleaner.

Professional style.

Without with:

You must manually close.

Easy to forget.

read()

Reads entire file.

Example:

```
file.read()
```

Reading Line by Line

Example:

```
with open("notes.txt") as file:  
    for line in file:  
        print(line)
```

Useful for large files.

readlines()

Returns list of lines.

Example:

```
lines = file.readlines()
```

Now:

```
[  
    "hello\n",  
    "world\n"  
]
```

Writing Files

Example:

```
with open("notes.txt", "w") as file:  
    file.write("Hello")
```

Modes:

Read:

"r"

Write:

"w"

Append:

"a"

Difference:

w:

overwrite existing content

a:

add to end

VERY important.

Exceptions

Programs crash when errors happen.

Exceptions let us handle errors.

Example crash:

10 / 0

Error:

division by zero

Handle safely:

```
try:
    print(10 / 0)
except ZeroDivisionError:
    print("Cannot divide by zero")
```

Meaning:

TRY risky code

If error happens

EXCEPT handles it

Why Exceptions Matter

Without exception:

program crashes

With exception:

program survives

Professional behavior.

Common Exceptions

Division:

`ZeroDivisionError`

Missing file:

`FileNotFoundError`

Wrong conversion:

`ValueError`

Wrong key:

`KeyError`

Wrong index:

`IndexError`

try / except / else / finally

Full structure:

```
try:
    risky code
except:
    handle error
```

```
else:
    if no error
finally:
    always runs
```

Example:

```
try:
    num = int(input("Number: "))
except ValueError:
    print("Invalid input")
else:
    print("Valid")
```

JSON

Used to store structured data.

Example:

```
{
    "name": "Alex",
    "age": 20
}
```

Python supports JSON.

Save JSON:

```
import json
```

Write:

```
json.dump(data, file)
```

Read:

```
json.load(file)
```

Example:

```
import json

data = {"name": "Alex"}
```



```
with open("data.json", "w") as file:  
    json.dump(data, file)
```

Read:

```
with open("data.json") as file:  
    data = json.load(file)
```

2. SYNTAX & MECHANICS

Open:

```
open(filename, mode)
```

With:

```
with open(...) as file:
```

Read:

```
file.read()
```

Write:

```
file.write()
```

Try:

```
try:
```

Catch:

```
except ErrorType:
```

JSON:

```
json.dump()  
json.load()
```

PRACTICAL EXAMPLES

File Write

```
with open("hello.txt", "w") as file:  
    file.write("Python")
```

File Read

```
with open("hello.txt") as file:  
    print(file.read())
```

Exception

```
try:  
    x = 10 / 0  
except ZeroDivisionError:  
    print("Error handled")
```

JSON Save

```
import json  
  
user = {"name": "Surjyo"}  
  
with open("user.json", "w") as file:  
    json.dump(user, file)
```

HIGH-YIELD PRACTICE

1. Create text writer.
 2. Create file reader.
 3. Append notes.
 4. Build safe calculator.
 5. Handle invalid age input.
 6. Build JSON user storage.
 7. Build mini login system using file.
-

COMMON MISTAKES

WRONG:

```
open("file.txt")
```

without closing.

Better:

```
with open()
```

WRONG:

```
int("hello")
```

ValueError.

Handle with try.

PRO TIPS

Master deeply:

- classes
- objects
- constructor
- self
- attributes
- methods
- inheritance
- file reading
- file writing
- exceptions
- JSON

This is real-world Python.

PYTHON MASTERY NOTES

PART 6 — CHAPTER 11 + PROFESSIONAL PYTHON THINKING

Based on *Python Crash Course*

This part is what separates:

someone who writes code
from
someone who writes reliable software

Beginners write code that "works once."

Professionals write code that can be tested, trusted, maintained, and improved.

This chapter is extremely important.

CHAPTER 11 — TESTING YOUR CODE

1. CONCEPT BREAKDOWN

What is Testing?

Testing means checking whether your code works correctly.

Simple meaning:

Instead of guessing:

"I think my code works"

You prove:

"Yes, my code works"

Example:

Calculator:

```
def add(a, b):  
    return a + b
```

Looks correct.

But what if:

- wrong numbers?
- negative numbers?
- decimal numbers?
- unexpected input?

Testing checks this.

Why Testing Matters

Without testing:

- bugs stay hidden
- apps fail
- users get errors
- updates break old code

With testing:

- confidence increases
 - debugging becomes easier
 - changes become safer
-

Think:

Engineer building bridge.

Would they guess?

No.

They test.

Software is same.

What is a Bug?

Bug = mistake in code.

Example:

```
def subtract(a, b):  
    return a + b
```

Function says subtract.

But does addition.

Bug.

Manual Testing

Testing by yourself.

Example:

```
print(add(2, 3))
```

Good for small checks.

Bad for large projects.

Slow.

Easy to miss things.

Automated Testing

Python tests code automatically.

Better.

Fast.

Repeatable.

Reliable.

Unit Testing

Test one small unit.

Usually one function.

Example:

Testing only:

```
add()
```

Not whole application.

Just one part.

Think:

Testing one Lego block before building whole castle.

Assertions

Assertion = expected truth.

Example:

```
assert add(2, 3) == 5
```

Meaning:

If result is not 5 → error.

unittest Module

Python built-in testing framework.

Lets you create formal tests.

Example:

```
import unittest
```

Test Case

A test case checks one behavior.

Example:

"Does add(2,3) return 5?"

Test Class

Group related tests.

Example:

Calculator tests together.

First Real Test

Function file:

```
def get_full_name(first, last):  
    return first + " " + last
```

Test:


```
import unittest
class NameTest(unittest.TestCase):

    def test_full_name(self):
        result = get_full_name("Alex", "Smith")
        self.assertEqual(result, "Alex Smith")
```

Meaning:

Create test class.

Create test method.

Run function.

Compare expected vs actual.

Naming Rules

Test methods should start with:

test_

Example:

test_login

Because test runner recognizes them.

assertEqual()

Checks equality.

Example:

```
self.assertEqual(actual, expected)
```

If different → fail.

Common Assertions

Equal:

```
assertEqual()
```

True:

```
assertTrue()
```

False:

```
assertFalse()
```

Contains:

```
assertIn()
```

Not contains:

```
assertNotIn()
```

Raises error:

```
assertRaises()
```

Examples:

```
self.assertTrue(is_even(4))  
self.assertFalse(is_even(5))
```

Testing Exceptions

Example:

```
with self.assertRaises(ValueError):
```

Meaning:

Error MUST happen.

Example:

```
with self.assertRaises(ValueError):  
    int("hello")
```

Passes because ValueError happens.

Multiple Tests

Example:

```
def test_add():  
def test_subtract():
```

Good design.

Small focused tests.

Why Small Tests?

Easy debugging.

If one fails, problem is obvious.

2. SYNTAX & MECHANICS

Import:

```
import unittest
```

Create class:

```
class MyTest(unittest.TestCase):
```

Test method:

```
def test_example(self):
```

Equality:

```
self.assertEqual(a, b)
```

True:

```
self.assertTrue(condition)
```

Raises:

```
with self.assertRaises(ValueError):
```

Run:

```
unittest.main()
```

PRACTICAL EXAMPLES

Example 1 — Calculator Test

Calculator:

```
def add(a, b):  
    return a + b
```

Test:

```
import unittest  
class CalcTest(unittest.TestCase):  
  
    def test_add(self):  
        result = add(2, 3)  
        self.assertEqual(result, 5)
```

Example 2 — Even Checker

Function:

```
def is_even(n):  
    return n % 2 == 0
```

Test:

```
self.assertTrue(is_even(4))
self.assertFalse(is_even(5))
```

Example 3 — Error Test

```
with self.assertRaises(ZeroDivisionError):
    10 / 0
```

HIGH-YIELD PRACTICE

1. Test calculator.
 2. Test square function.
 3. Test even checker.
 4. Test login validator.
 5. Test string formatter.
 6. Test file reader.
 7. Test JSON loader.
-

COMMON BEGINNER MISTAKES

WRONG:

```
def full_name():
```

No parameters.

But test passes arguments.

Mismatch.

WRONG:

```
assertEqual()
```

Without `self`.

Correct:

```
self.assertEqual()
```

PROFESSIONAL PYTHON THINKING

Now I teach what the book gives indirectly—but what professionals actually do.

This is extremely important.

CLEAN CODE THINKING

Bad code:

```
x = 10
y = 20
z = x + y
```

What is x?

What is y?

Unclear.

Better:

```
price = 10
tax = 20
total = price + tax
```

Readable.

Meaningful Names

Bad:

```
a
b
temp
```

Good:

```
student_name  
total_price  
user_age
```

Small Functions

Bad:

One giant function doing everything.

Good:

Split tasks.

Example:

```
validate_user()  
calculate_total()  
save_data()
```

Cleaner.

DRY Principle

DRY = Don't Repeat Yourself

Bad:

Same code repeated 20 times.

Better:

Use function.

Debugging Mindset

When code fails:

DON'T PANIC.

Think:

1. What error?
2. Which line?
3. What variable values?
4. What did I expect?
5. What actually happened?

Read Error Messages

Example:

`NameError`

Means:

variable missing

Example:

`TypeError`

Means:

wrong data type operation

Example:

`IndexError`

Means:

bad list index

Example:

`KeyError`

Means:

dictionary key missing

Example:

`ValueError`

Means:

bad value conversion

Problem Solving Mindset

When solving coding problems:

Understand input

Example:

What data comes in?

Understand output

Example:

What must be returned?

Break into steps

Example:

Need palindrome checker?

Steps:

- get string

- reverse string
 - compare
-

Write Pseudocode

Before coding:

Example:

```
get input
reverse input
compare
show result
```

Then code.

Professional habit.

Interview Thinking

For coding interviews:

Always know:

- lists
 - dictionaries
 - loops
 - conditions
 - functions
 - strings
 - OOP basics
 - exceptions
 - file handling
-

Common interview patterns:

Reverse string

Find duplicates

Count frequency

Palindrome

Prime checker

Two sum logic

Max/min finder

Dictionary counting

Pythonic Thinking

Bad:

```
for i in range(len(items)):
```

Often better:

```
for item in items:
```

Cleaner.

Bad:

Long ugly loops

Better:

List comprehension

When readable.

Use built-ins:

```
sum()  
max()  
min()  
sorted()
```

`len()`

These are fast and clean.

Professional Habits

Always:

write readable code

test code

handle errors

use meaningful names

break big problems into small parts

practice consistently

read documentation

build projects

MINI PROFESSIONAL CHALLENGES

Build these yourself:

1. ATM machine
 2. Student result manager
 3. Bank account system
 4. Contact manager
 5. Inventory manager
 6. Quiz app
 7. Login/signup system
 8. Expense tracker
 9. File organizer
 10. JSON-based user storage
-

BIG TRUTH

Reading notes is NOT enough.

To become strong:

Type code.

Break code.

Fix code.

Build projects.

Repeat.

That is how programmers are made.

END OF CORE PYTHON MASTERY

NEXT WE MOVE TO:

PROJECT SECTION FROM THE BOOK

(Alien Invasion, Data Visualization, APIs, Django Web Apps)

OR

I can create your **ULTIMATE ADVANCED PYTHON MASTER NOTE**
that goes beyond the book and teaches:

- decorators
- generators
- iterators
- lambda
- map/filter/reduce
- regex
- context managers
- dataclasses
- comprehensions deep dive
- advanced OOP
- dunder methods

- package structure
- virtual environments concepts
- pip
- concurrency basics
- async intro
- professional project architecture

That would push you much closer to pro level.